

Algorithmique avancée et programmation

CERI L2 Informatique

BAROUDI TOUFIK

Programme

- Introduction
- Chapitre1: Les arbres
- Chapitre 2: Le Tas (Heap)
- Chapitre 3: Les algorithmes de tri
- Chapitre 4: La complexité algorithmique
- Chapitre 5: Backtracking,
- Chapitre 6: Programmation dynamique
- Chapitre 7: Les tables de hachage



Introduction

Définition:

Un **algorithme** se définit comme une suite finie d'opérations élémentaires pour la résolution d'un problème donné.

Définition:

L'**algorithmique** est l'ensemble des méthodes et des techniques permettant de **concevoir** et d'**analyser** un algorithme.

Introduction

Définition:

L'**analyse** d'un algorithme consiste :

- ❖ À prouver que la méthode proposée résout bien le problème donné en entrée (**preuve de l'algorithme**),
- ❖ À évaluer le temps d'exécution et l'espace mémoire requis (**complexité en temps et en espace**).

Définition:

Un **programme** informatique est le codage d'un algorithme dans un langage informatique donné (C, C++, Java, Python,...).

Introduction



Ecriture des algorithmes:

- Un algorithme doit pouvoir être programmé dans **n'importe quel langage informatique**.
- Nous décrirons les algorithmes par des “pseudo-codes”.
- Les implémentations se feront en **C++ en TP**.

Introduction

Plan du cours:

	Chapitre	CM	TD	TP
1	Arbres	2	2	2
2	Tas	1	1	1
3	Tris	1	1	1
	CC1			
4	Complexité algorithmique	1	1	1
5	Backtracking	1	1	1
6	Programmation dynamique	1	2	2
7	Tables de hachage	1	1	1
	CC2			

TP Codes C++:

localement ou sur Caseine

Note Algorithmique avancée:

$0,4 * \text{CC1}(\text{écrit}) + 0,4 * \text{CC2}(\text{écrit}) + 0,2 * \text{TPs}$



Chapitre 1: **Les arbres**

Chapitre1: Les arbres

Introduction

Définition:

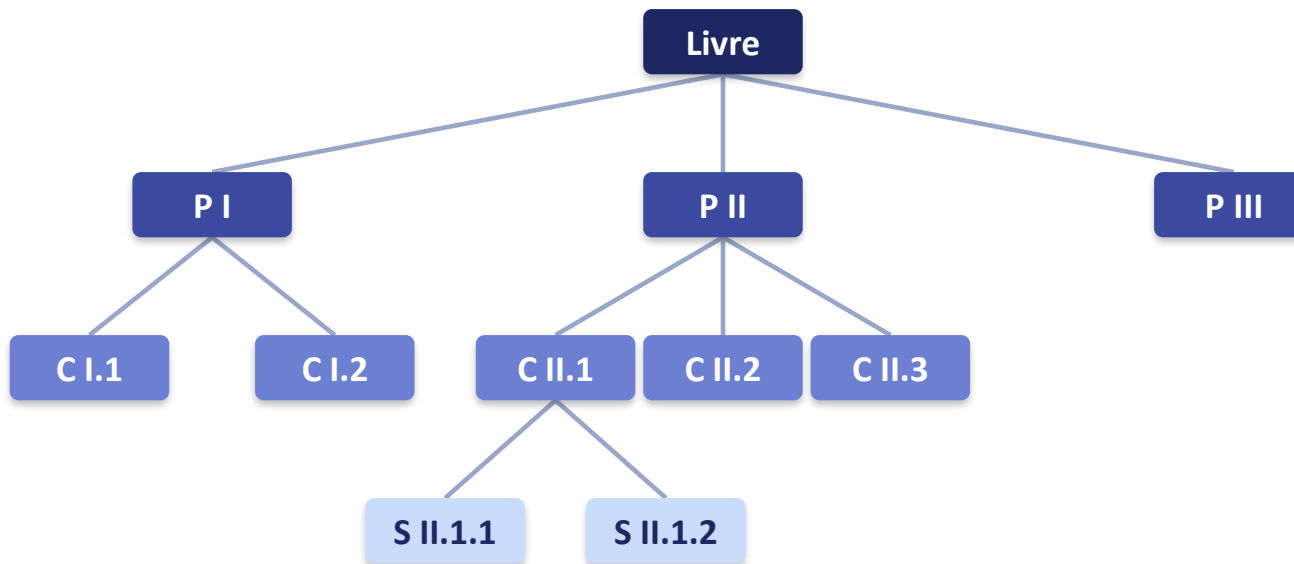
Un **arbre** est un ensemble de nœuds organisés de façon hiérarchique à partir d'un nœud appelé racine.

Introduction

Leur usage est multiple, car il capte l'idée de hiérarchie.

Exemple:

- ❖ Découpage d'un livre en parties, chapitres, sections, paragraphes...,

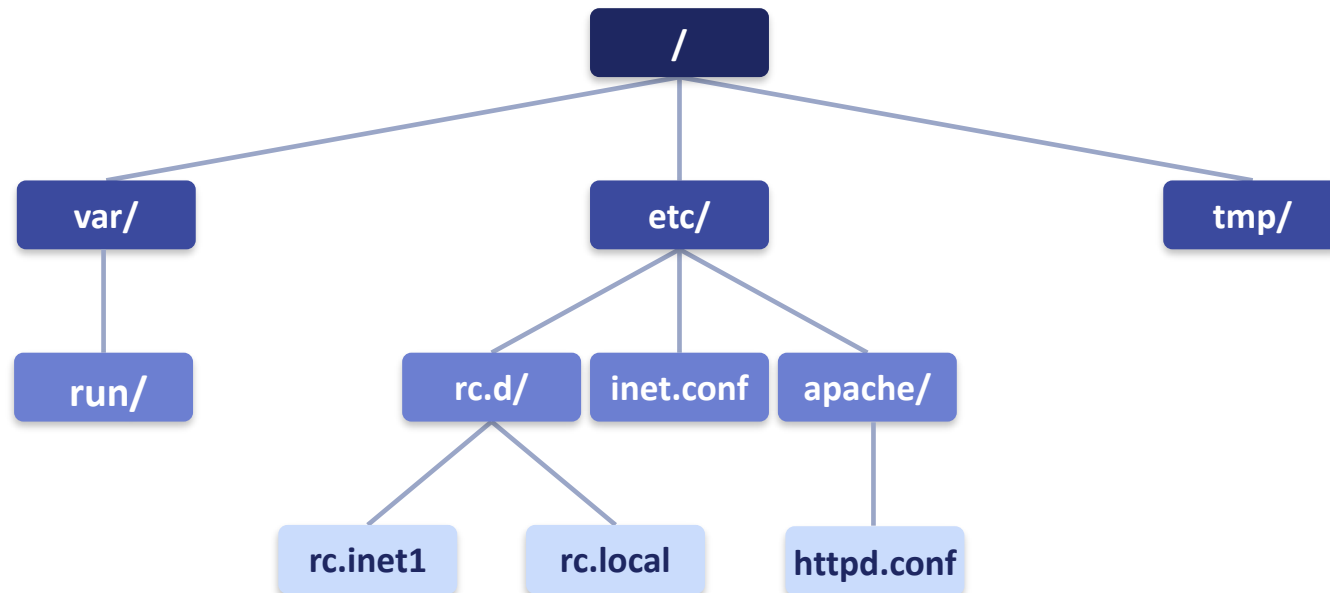


Introduction

Leur usage est multiple, car il capte l'idée de hiérarchie.

Exemple:

❖ Hiérarchies de fichiers.



Chapitre1: Les arbres

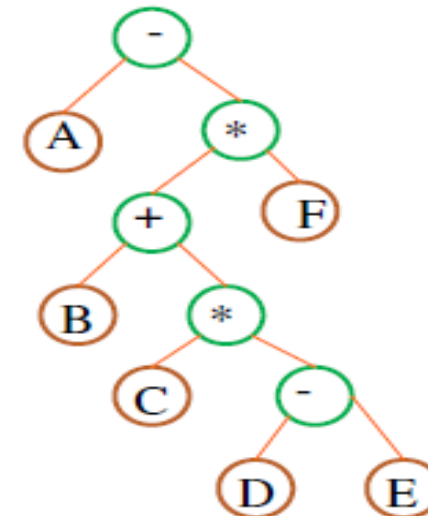
Introduction

Leur usage est multiple, car il capte l'idée de hiérarchie.

Exemple:

❖ Expressions arithmétiques.

- L'expression $A - (B + C * (D - E)) * F$ se représente facilement par un arbre où apparaît clairement la priorité des opérations:

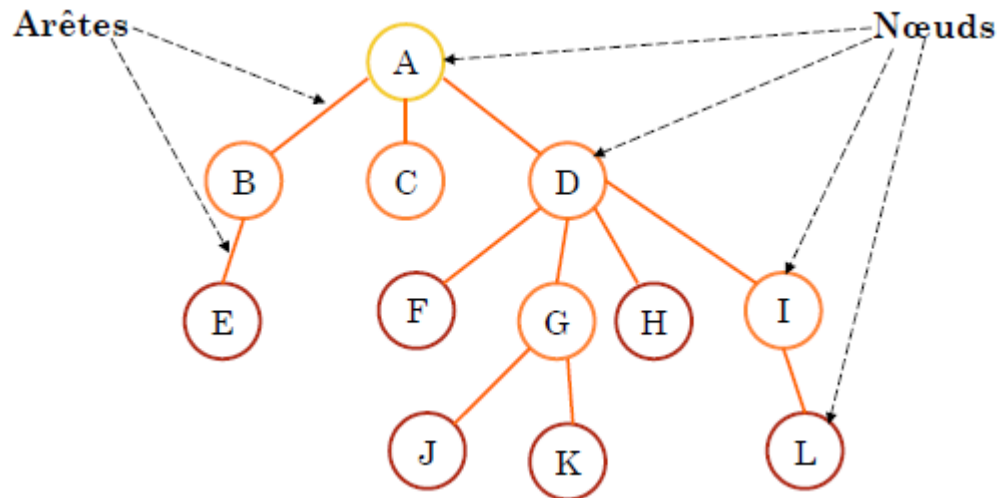


Chapitre1: Les arbres

Définitions & Terminologies



Un arbre est une structure de données (souvent dynamique) représentant un ensemble de valeurs organisées hiérarchiquement (non linéaire). Chaque valeur est stockée dans un **nœud**. Les nœuds sont connectés entre eux par des **arêtes** qui représentent des relations parent/fils.



Chapitre1: Les arbres

Définitions & Terminologies



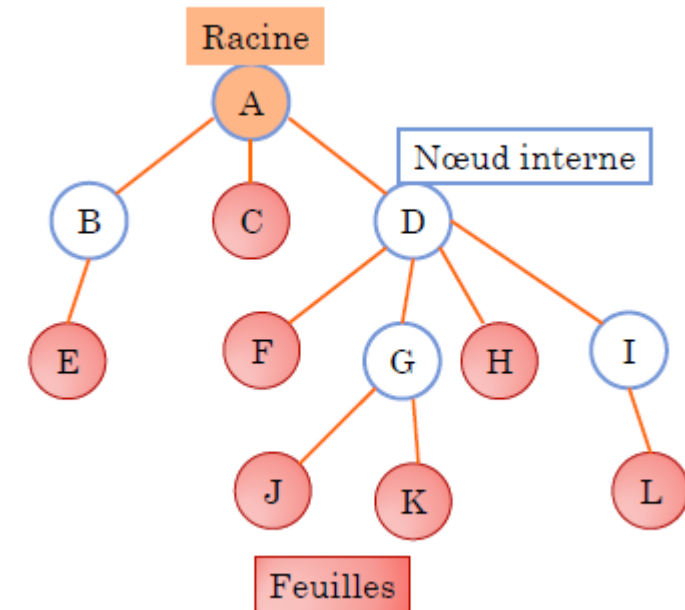
Racine: est le nœud qui n'a pas de **prédécesseur** (parent) et possède zéro ou plusieurs **fils**. La racine constitue la caractéristique d'un arbre.



Feuille : est un nœud qui n'a pas de **successeur** (fils). Une feuille est aussi appelée un nœud externe.



Noeud interne : est tout nœud qui admet au moins un successeur (fils).



Chapitre1: Les arbres

Définitions & Terminologies



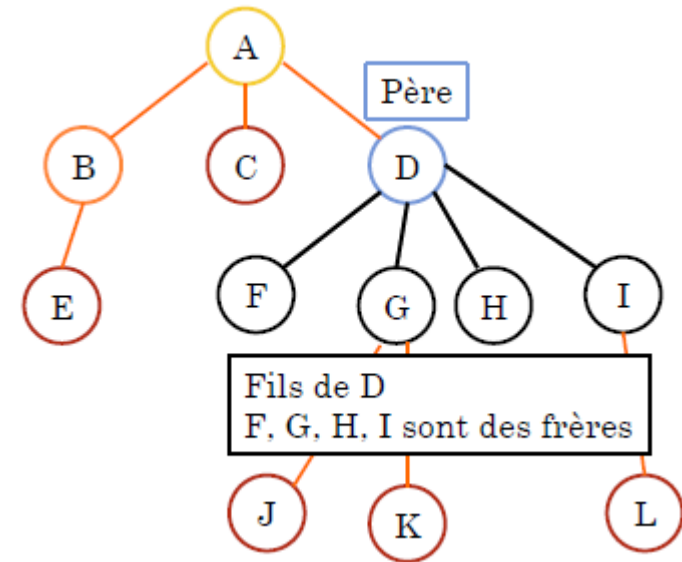
Père : est un nœud qui admet au moins un successeur (fils). Dans l'exemple, D est le père des nœuds F, G, H et I.



Fils d'un nœud : sont ses successeurs. Dans l'exemple, F, G, H, et I sont les fils du nœud D.



Frères : sont les successeurs ou les fils issus d'un même nœud (parent direct). Dans l'exemple, F, G, H et I sont des frères.



Définitions & Terminologies

Définition:

Un arbre, enraciné en r , à **ramification bornée** peut être défini récursivement comme suit:

$$A = \emptyset \text{ ou } A = \langle r, F_1, F_2, \dots, F_k \rangle$$

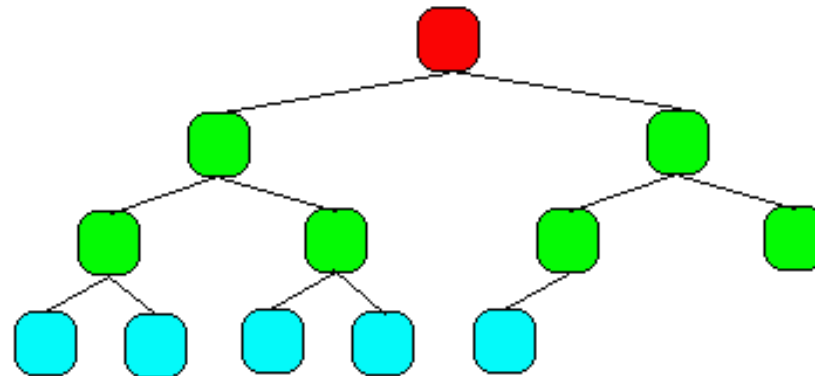
où ' r ' est un nœud appelé racine et $F_1, F_2, \dots, F_k$ sont des arbres disjoints appelés **sous-arbres** de A .

Les nœuds contiennent plusieurs champs dont **une étiquette** (identifiant) nommée "**cle**" (**r.cle**).

Définitions & Terminologies

Arbre binaire, $K=2$:

- F1 : sous-arbre gauche, sa racine est le fils gauche de r.
- F2 : sous-arbre droit, sa racine est le fils droit de r.
- Noeud simple : noeud interne n'ayant qu'un fils.
- Noeud double : noeud interne ayant deux fils.



Représentation des AB

Noeud	
cle	un identifiant
pere	adresse du nœud père
fg	adresse du nœud dit fils gauche
fd	adresse du nœud dit fils droit

Pour un nœud x , $x.cle$, $x.pere$, $x.fg$, $x.fd$ désignent les champs.

Chapitre1: Les arbres

Codage des AB en c++

```
class noeud
{
    friend class AB;
    int cle ;
    noeud * fg ;
    noeud * fd ;
    noeud * pere ;
public :
    noeud(int x) ;
    ~noeud() ;
    ...
} ;
```

```
class AB
{
    noeud * r ;
public :
    AB(noeud * x) ;
    ~AB() ;
    ...
} ;
```

“friend class AB” : toutes les méthodes de la classe AB peuvent accéder aux attributs privés de “noeud”.

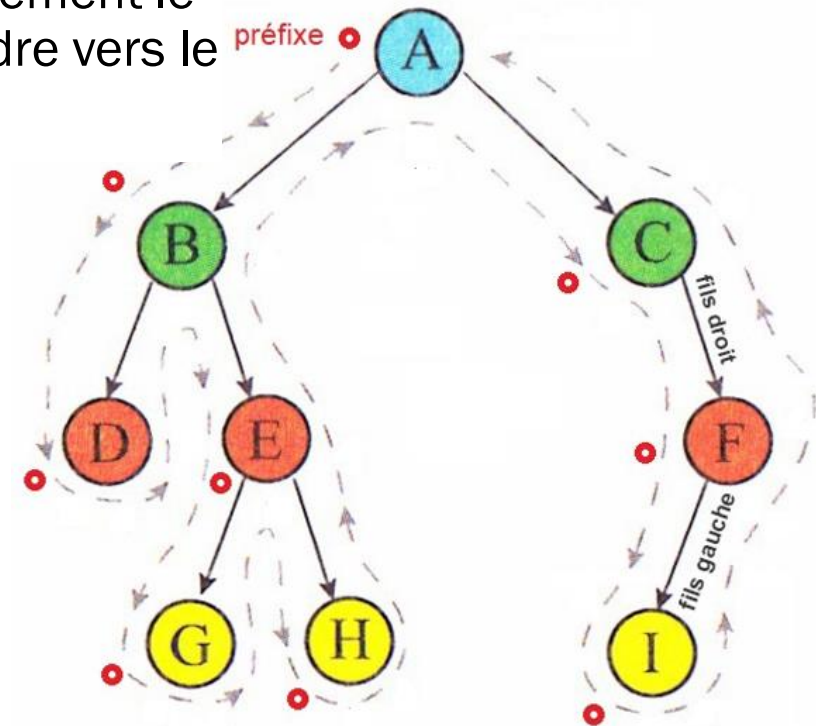
Chapitre1: Les arbres

Parcours dans un AB

Parcours Préfixe:

On part de la racine puis on visite son fils gauche (et éventuellement le fils gauche de celui-ci, etc.) avant de remonter et de redescendre vers le fils droit.

```
Algorithme: prefixe(r)  
Si (r ≠ null) alors  
    affichage r.cle;  
    prefixe(r.fg);  
    prefixe(r.fd);  
Fin
```



A B D E G H C F I

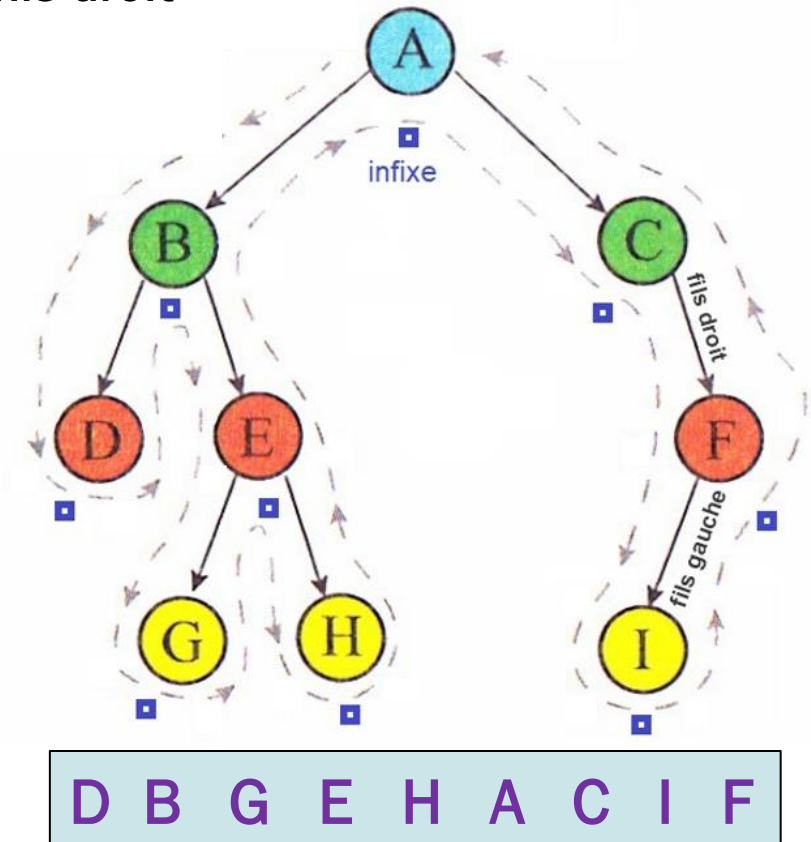
Chapitre1: Les arbres

Parcours dans un AB

Parcours Infixe:

Chaque nœud est visité après son fils gauche mais avant son fils droit.

```
Algorithme: infixe(r)
Si (r≠ null) alors
    infixe(r.fg);
    affichage r.cle;
    infixe(r.fd);
Fin
```



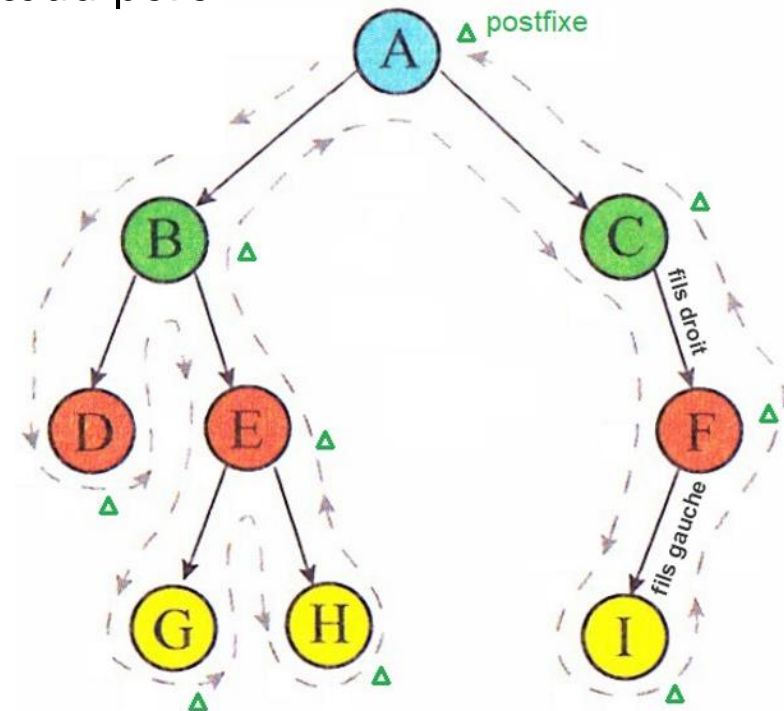
Chapitre1: Les arbres

Parcours dans un AB

Parcours Postfixe:

On part de la **feuille la plus à gauche**, et on ne remonte à un nœud père que si ses fils ont tous été visités.

```
Algorithme: postfixe(r)
Si (r≠ null) alors
    postfixe(r.fg);
    postfixe(r.fd);
    affichage r.cle;
Fin
```



D G H E B I F C A

Chapitre1: Les arbres

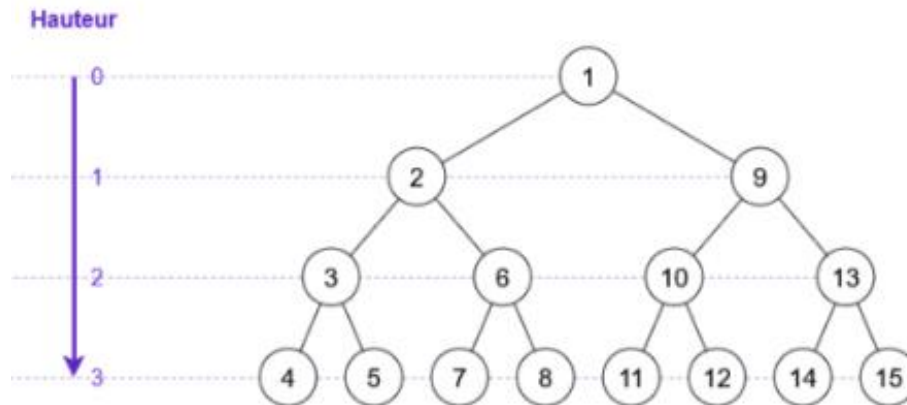
Hauteur

Définition: On appelle **hauteur** h , ou **profondeur**, d'un nœud x d'un arbre binaire B de racine r , la fonction définie récursivement comme suit :

- $h(r) = 0$
- $h(x) = 1 + h(y)$, si y est le père de x

Définition: On appelle **hauteur** h , ou **profondeur**, d'un arbre binaire B , la valeur:

- $h(B) = \max \{ h(x) \mid x \text{ est un nœud de } B \}$



Hauteur de l'arbre : $h(B) = 3$

Chapitre1: Les arbres

Hauteur

Définition:

Un arbre binaire B dont tous les nœuds (autre que les feuilles) sont doubles est dit **complet**.

Théorème:

Un arbre binaire B est complet ssi il a 2^i nœuds de hauteur i pour i allant de 0 à $h(B)$. Le nombre total de ses nœuds est $2^{h(B)+1} - 1$

Théorème:

Soit B un arbre binaire de n nœuds et de hauteur $h(B)$. On a:

$$\log_2(n) \leq h(B) \leq n - 1$$